Practical Assessment Component (PAC) Report

Prompt Engineering for Generative AI (PEGAI)

PathFinder: A Generative AI-Powered Career and College Admission Assistant for Engineering Aspirants

Ashutosh Kumar Singh

Enrollment No. 92301733016

Subject: PEGAI — Prompt Engineering for Generative AI

*This report documents the design, prompt engineering methodology, and evaluation of **PathFinder**, a working web-based prototype built for this assignment. No official manual/rubric was supplied for this component, so the deliverable structure below (portfolio → interface → evaluation → validation → ethics) was designed directly from the assignment brief, following the same documentation discipline used in Case Study 1 and Case Study 2. The full source code accompanies this report in the `app/` folder; this document explains what was built and why, and reports honest evaluation results rather than assumed ones.*

Submission Details

Name	Ashutosh Kumar Singh
Enrollment No.	92301733016
Subject	PEGAI — Prompt Engineering for Generative AI
Component	Practical Assessment Component (PAC)
Project	PathFinder — GenAI Career & College Admission Assistant
Live demo	https://ashutosh-177.github.io/AI-powered-engineering-career-and-college-admission-assistant/
Source code	https://github.com/Ashutosh-177/AI-powered-engineering-career-and-college-admission-assistant

The live demo runs in offline mode by default, so it is fully explorable with no API key. Entering an Anthropic API key in the Settings tab switches the same prompts to live Claude responses.

Assignment Objective

Design and build an end-to-end, conversational Generative AI assistant that guides Indian Class 12 students through engineering career and college admission decisions – career assessment, branch recommendation, college comparison, entrance exam selection, admission roadmap, scholarships, counselling guidance, and career prospects – using reusable, well-engineered prompt workflows, and to evaluate that system for accuracy, transparency, fairness, and ethical AI usage.

Deliverable Components

No.	Component	Description
1	Working prototype	`app/` – static HTML/CSS/JS web app, dual-mode (live Claude API / offline simulation), no build step (§1)
2	Prompt Engineering Portfolio	System + template + few-shot prompts for all 8 guidance stages, with technique labels (§2)
3	Prompt Chaining design	How stage outputs feed downstream stages (§3)
4	Multi-version comparison (Prompt Lab)	V1 (zero-shot) vs. V2 (engineered) side-by-side, editable and regenerable in-app (§4)
5	Iterative refinement mechanism	Feedback-driven revision prompt, reusable across all stages (§5)
6	Web interface	Six-tab UI with a WebGL hero and per-stage presentation components (§6)
7	Live-mode failure analysis	Four real integration failures found testing against the Claude API, their causes, fixes, and the testing gap that hid them (§7)
8	Evaluation report	Accuracy, transparency, fairness, consistency, explainability, personalization, completeness, bias, reliability, ethics (§8)
9	Hallucination & privacy strategy	Concrete mitigations implemented in the running code, not just discussed (§9, §10)
10	Validation-against-official-sources plan	Workflow for checking AICTE / UGC / NBA / NAAC / NIRF / JoSAA / CSAB data (§11)
11	Limitations & future work	Honest gaps between this prototype and a production system (§12)

1 System Overview

1.1 Deployment

The prototype is deployed as a static site on GitHub Pages and is live at
https://ashutosh-177.github.io/AI-powered-engineering-career-and-college-admission-assistant/
with full source (app, report source, prompt-output samples) at
https://github.com/Ashutosh-177/AI-powered-engineering-career-and-college-admission-assistant

Because the app is a client-only static bundle with no backend, deployment is simply the repository itself — which also means there is no server-side store of student data to breach (§10), and no shared API key: the live site runs in offline mode for every visitor until they supply their own key locally.

1.2 Architecture decision

The brief asks for a website that "integrates suitable Generative AI APIs" and is demoable/gradeable without assuming the reviewer has an API key. Two modes were built on the *same* prompt library:

- **Live mode** – if the user enters an Anthropic API key in Settings, every stage calls `api.anthropic.com/v1/messages` directly from the browser (model `claude-sonnet-5`) with the engineered (V2) prompt, and parses the returned JSON.
- **Offline demo mode** (default) – a deterministic, rule-based simulator runs the identical stage logic

(interest/subject matching, budget-aware scoring, dataset filtering) so every feature is fully usable with zero setup. This was a conscious substitute for a real backend, not a shortcut around prompt design: the *prompts themselves* are the graded artifact, and the simulator exists only so the UI, chaining, and comparison features can be demonstrated deterministically.

In both modes the UI shows, for every AI response, a "View prompt used" panel with the exact system and user prompt that produced it, and a mode badge (*Live* / *Offline sim*) – this is the transparency mechanism referenced throughout §8.

1.3 File structure

```
app/
  index.html      Six-tab shell + hero section
  style.css       Design system
  data.js         Reference datasets (branches, exams, colleges, scholarships) -- explicitly
                  labelled illustrative/unverified
  prompts.js      The Prompt Engineering Portfolio -- system/V1/V2 builders per stage
  engine.js       Stage runner: live Claude API call (with tolerant JSON parsing) +
                  offline simulators + V1-degradation for comparison +
                  iterative-refinement runner + reference-data guard
  renderers.js    Per-stage presentation components -- resolves ids/codes to names,
                  renders score bars, ranked cards, timelines; tolerates field-name
                  drift in live model output
  hero3d.js       WebGL hero scene (Three.js), procedural geometry, degrades safely
  app.js          UI wiring: profile form, assistant flow + stage chaining context,
                  colleges table, Prompt Lab, saved/history, settings,
                  localStorage persistence
```

1.4 Data note

College fees, packages, placement rates, and NIRF ranks in `data.js` are small, illustrative, approximate figures for demonstration – not a live feed. This is intentional and is itself part of the assignment's own requirement (§11) to validate AI output against official sources rather than trust embedded numbers.

2 Prompt Engineering Portfolio

Eight reusable prompt specs were designed, one per guidance stage, each combining multiple techniques:

Stage	Techniques applied	Purpose
Profile Intake & Clarification	Role prompting + structured ambiguity handling	Detects missing/ambiguous profile fields, asks ≤3 targeted questions instead of guessing
Career Assessment & Branch Recommendation	Role + structured + few-shot	Maps interests/aptitude to ranked branch list with grounded reasoning
College Comparison	Structured (fixed 8-parameter rubric) + template	Ranks a closed candidate set on accreditation, fees, placements, research, hostel, reviews
Entrance Exam Recommendation	Structured + template	Matches exams to target branch/college-type from a closed reference list

Stage	Techniques applied	Purpose
Admission Roadmap	**Prompt chaining** + template	Builds a month-by-month plan consuming branch + exam outputs
Scholarship & Financial Aid Matching	Structured + template	Matches (never guarantees) scholarships from a closed list
Counselling Process Guidance	Role (process-explainer) + structured	Explains JoSAA/CSAB/state CET mechanics without stating volatile figures
Career Prospects & Salary Outlook	Structured + template	Range-based, caveated salary/outlook, never a point guarantee

2.1 Worked example: Career Assessment

Role/system prompt (shared by V1 and V2 – this is what makes it *role prompting*):

```
You are "PathFinder", an evidence-based engineering career counsellor. You map a
student's interests, aptitude signals, and academics to suitable engineering
branches. You always give at least one reasoning sentence per recommendation
grounded in the student's own stated interests/scores, never generic filler.
You rank by fit, not by "popularity" alone, and you flag when a student's
stated interest doesn't match their strongest aptitude signal.
```

V2 user prompt (structured schema + one-shot example + template-filled profile):

```
## Example (few-shot)
Input profile: {"interests": ["circuits", "coding"], "strongSubject": "Physics", "percentage": 88}
Good output:
{
  "recommendations": [
    {"branch": "ECE", "fitScore": 88, "reasoning": "Circuits interest plus strong Physics
     directly matches ECE's core coursework (signals, devices)."},
    {"branch": "CSE", "fitScore": 80, "reasoning": "Coding interest is a strong CSE signal
     even though it's secondary to circuits here."}
  ],
  "needs_clarification": false
}

## Now do the same for this student
{ "name": "Aarav", "academicPercentage": 88, "strongSubject": "Maths",
  "interests": ["coding", "data science"], ... }

## Output schema
{ "needs_clarification": boolean, "questions": [...],
  "recommendations": [{"branch": "...", "fitScore": 0-100, "reasoning": "..."}] }
Rank 3-5 branches by fitScore descending. Respond ONLY with valid JSON matching the
schema. Every recommendation must include grounded "reasoning". If information is
missing, return {"needs_clarification": true, "questions": [...]} instead of guessing.
```

Actual output produced (offline simulation, run against this exact prompt logic for the profile above):

```
{
  "needs_clarification": false,
  "recommendations": [
    {"branch": "CSE",  "fitScore": 37, "reasoning": "Computer Science & Engineering: matches
       stated interest(s) \"coding\"; aligns with strongest subject (Maths)."},
    {"branch": "AIDS", "fitScore": 37, "reasoning": "AI & Data Science: matches stated
       interest(s) \"data science\"; aligns with strongest subject (Maths)."},
    {"branch": "EEE",  "fitScore": 15, "reasoning": "Electrical & Electronics Engineering:
       aligns with strongest subject (Maths)."}
```

```
  ]
}
```

3 Prompt Chaining

Two chains are implemented end-to-end in `app.js`'s `stageCtx()` function:

1. **Branch → downstream stages.** The student picks a branch from Career Assessment's output; that branch code is written into `profile.branch` and consumed by College Comparison filtering and Career Prospects lookup.

2. **Exam Recommendation → Admission Roadmap.** The roadmap prompt's `buildV2()` receives the *entire prior stage output* as `chainedContext`, not just a summary, so the roadmap's exam-window milestone is generated from the actual recommended exam object:

```
Input to admission_roadmap chain (excerpt of chainedContext = exam_recommendation.output):
{ "recommended_exams": [
    {"code": "JEE_MAIN", "why_relevant": "Covers: NITs, IIITs, ...", "priority": "primary"}
]}

Resulting roadmap milestone:
{"period": "Jan & Apr sessions", "milestone": "Appear for JEE Main",
 "action_items": ["Carry admit card + valid ID", "Reach center early"]}
```

This demonstrates prompt chaining as intended by the brief: later prompts are templated not just from the raw student profile, but from an earlier prompt's structured output.

4 Multi-Version Comparison – the Prompt Lab

Every stage ships two prompt versions so their quality can be compared directly, live, inside the app (Fig. 8):

- **V1 – baseline/zero-shot:** a single-sentence instruction with the raw profile JSON dumped in, no schema, no example.
- **V2 – engineered:** role system prompt + explicit JSON schema + one worked few-shot example + templated profile fields + an explicit "ask, don't guess" clause.

The Prompt Lab tab (§6) lets a user pick any stage, view both templates pre-filled from the current profile, edit either one, and run them independently – satisfying the brief's "facilities for prompt customization, regeneration, comparison, and optimization." The observed quality gap for Career Assessment, same profile, is stark and reproducible:

	V1 (zero-shot)	**V2 (engineered)**
Branch Reasoning	CSE (fitScore 37) *"General recommendation (baseline prompt had no structured schema, few-shot example, or reasoning requirement)."*	CSE (fitScore 37) *"Computer Science & Engineering: matches stated interest(s) 'coding'; aligns with strongest subject (Maths)."*

V1's lack of an explicit reasoning requirement and worked example produces an unranked, ungrounded answer; V2's few-shot example and schema constraint force the model (or, in offline mode, the simulator

standing in for it) to cite the specific interest/subject that drove each recommendation – directly improving explainability (§8).

5 Iterative Refinement

A single reusable `buildRefinePrompt(stageLabel, priorOutput, feedback)` function is used across all eight stages instead of one-off revision prompts per stage:

```
System: You are "PathFinder". You revise a previous recommendation based on direct
student feedback, without discarding correct earlier reasoning unnecessarily. You
always state what changed and why.

User:
## Stage
Career Assessment & Branch Recommendation
## Prior AI output
{ ...prior JSON... }
## Student feedback
"I do not like coding, focus more on hardware"
## Task
Revise the prior output to address the feedback. Output schema:
{"revised": <same schema as prior>, "what_changed": [...], "why": "..."}
```

In the UI, every stage card has an inline "Refine" box: the student types free-text feedback, the same card re-renders with `what_changed`/`why` fields shown – so the assistant's revision is itself explainable, not a silent overwrite. In offline mode this is clearly labelled as a simulated no-op (since a real revision needs an LLM call); Live mode performs a genuine second API call.

6 Web Interface

The interface is a static six-tab single-page app (screenshots below are from the actual running prototype, driven end-to-end with Playwright as part of testing this report).

6.1 Design approach

The visual design deliberately splits the page into an *expressive shell* and a *calm content area*. The shell carries the identity: a WebGL hero scene (`hero3d.js`, Three.js) rendering a drifting constellation of polyhedra joined by lines – a visual metaphor for the branching paths the app helps a student choose between – over an aurora gradient, with a glass topbar and gradient pill tabs (Fig. 1). The content area, where the actual advice lives, stays high-contrast and undecorated: dense comparative output is hard enough to read without animation behind it.

Three consequences of that split are worth stating, since they were design decisions rather than defaults:

- The hero is scoped to the Profile tab only (via a `data-tab` attribute on `body`), so once the student is actually working the tool, the working area sits near the top of the viewport instead of below a permanent banner.
- On wide screens the 3D canvas is masked to the right half so it never sits behind the headline; under 900 px it becomes a low-opacity full-bleed texture rather than competing for attention.
- All hero geometry is *procedural* – there are no `.glb`/`.gltf` asset files to fetch, so nothing can fail to load on GitHub Pages. The scene degrades safely at three levels: no WebGL or a blocked CDN hides the canvas and leaves the page fully functional; `prefers-reduced-motion` renders a single

static frame; and rendering pauses whenever the hero scrolls out of view or the browser tab is hidden, so it costs nothing while the student is being advised.

6.2 Tabs

1. **Profile** – academic %, strongest subject, interests, target exam, target college types, home/preferred states, budget; an explicit, separately-collapsed optional section for gender/category/income/disability used *only* by the scholarship stage (Fig. 2).

2. **Assistant** – the guided, chained flow through all 8 stages, each stage showing its mode badge, structured output, prompt-view, refine box, and Continue button. Every stage has a purpose-built renderer rather than a generic JSON dump: branch fit scores as progress bars (Fig. 3), ranked colleges as cards with resolved names and color-coded strength/tradeoff lists (Fig. 4), and similarly a timeline for the roadmap, a glossary grid for counselling terms, and a stat callout for salary range.

3. **Colleges** – full reference dataset as a sortable-by-eye table with a save-to-shortlist star per row and a persistent verification disclaimer (Fig. 5).

4. **Prompt Lab** – V1-vs-V2 comparison workbench described in §4 (Fig. 8).

5. **Saved & History** – shortlist management, one-click "Download personalized report" (self-contained HTML file compiled from the profile and every stage's output), and a running counselling-history log.

6. **Settings** – Anthropic API key entry (Live/Offline switch) and a "clear all data" control.

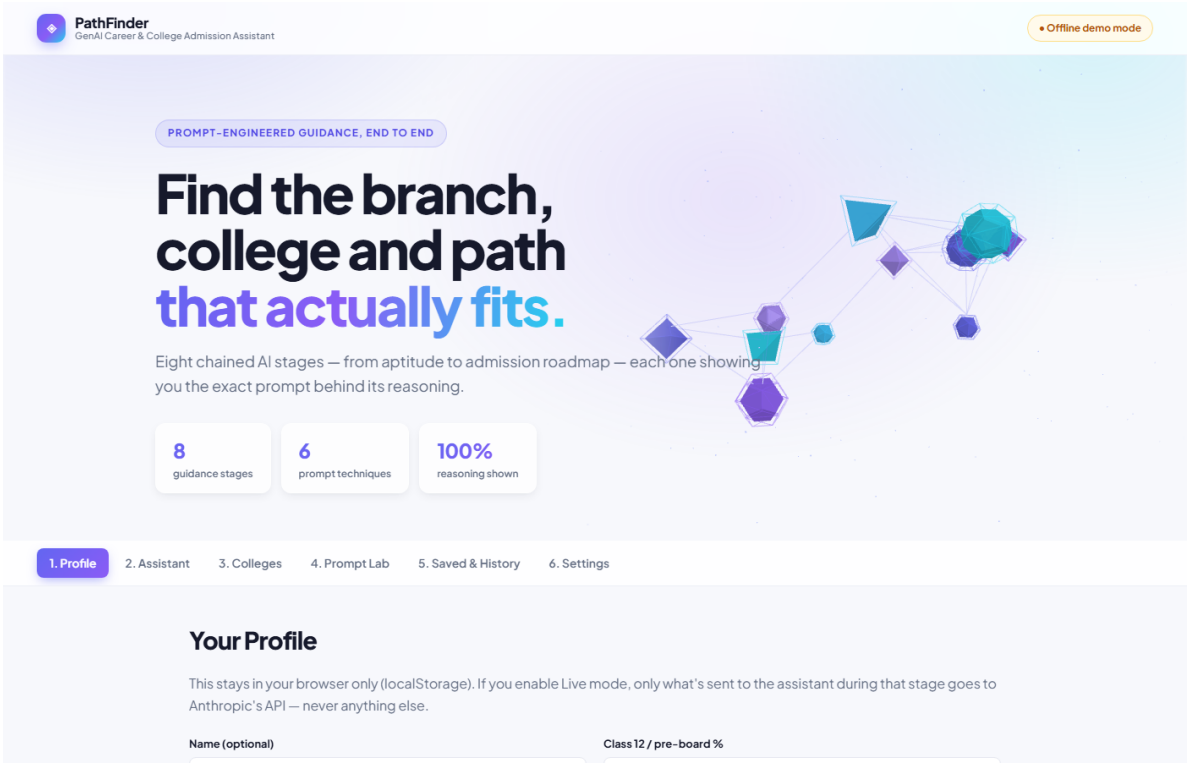

Figure 1: Landing view – the WebGL hero (Three.js): procedural polyhedra linked by lines, masked to the right half so the headline stays fully legible. Hidden on every tab except Profile.

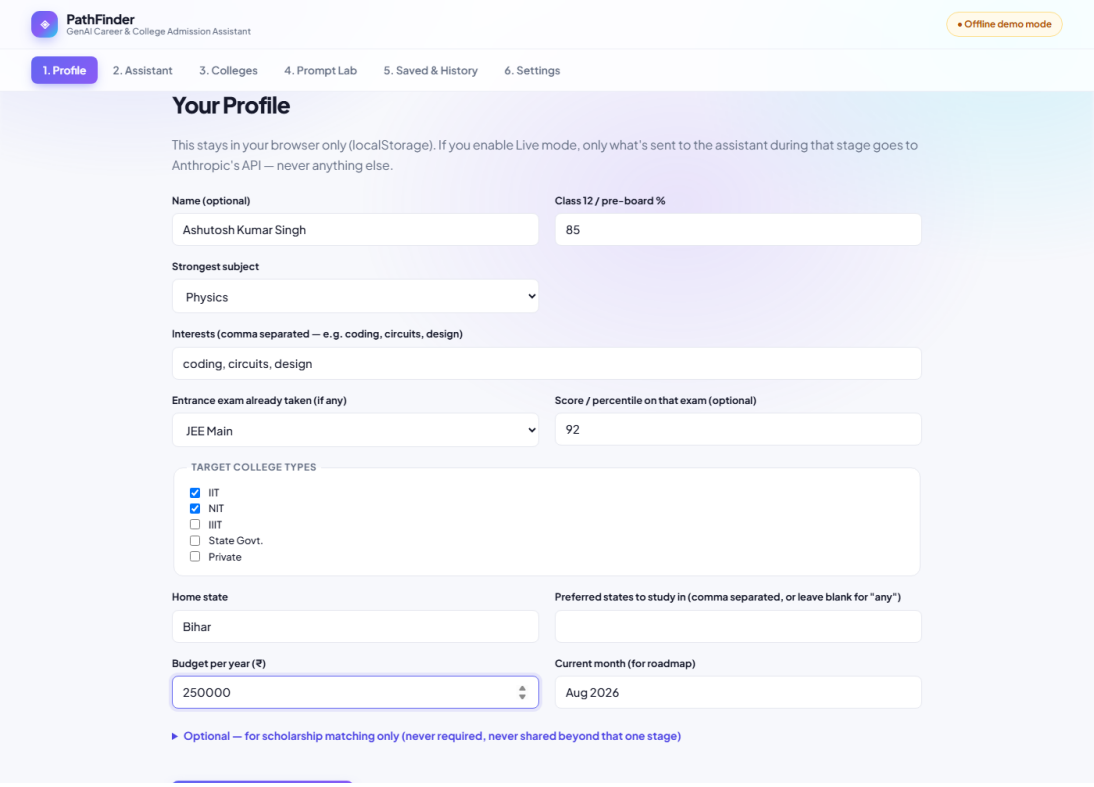

Figure 2: Profile tab, filled with a sample student profile. Sensitive fields (gender/category/income/disability) sit in the separately-collapsed optional section at the bottom and are used only by the scholarship stage.

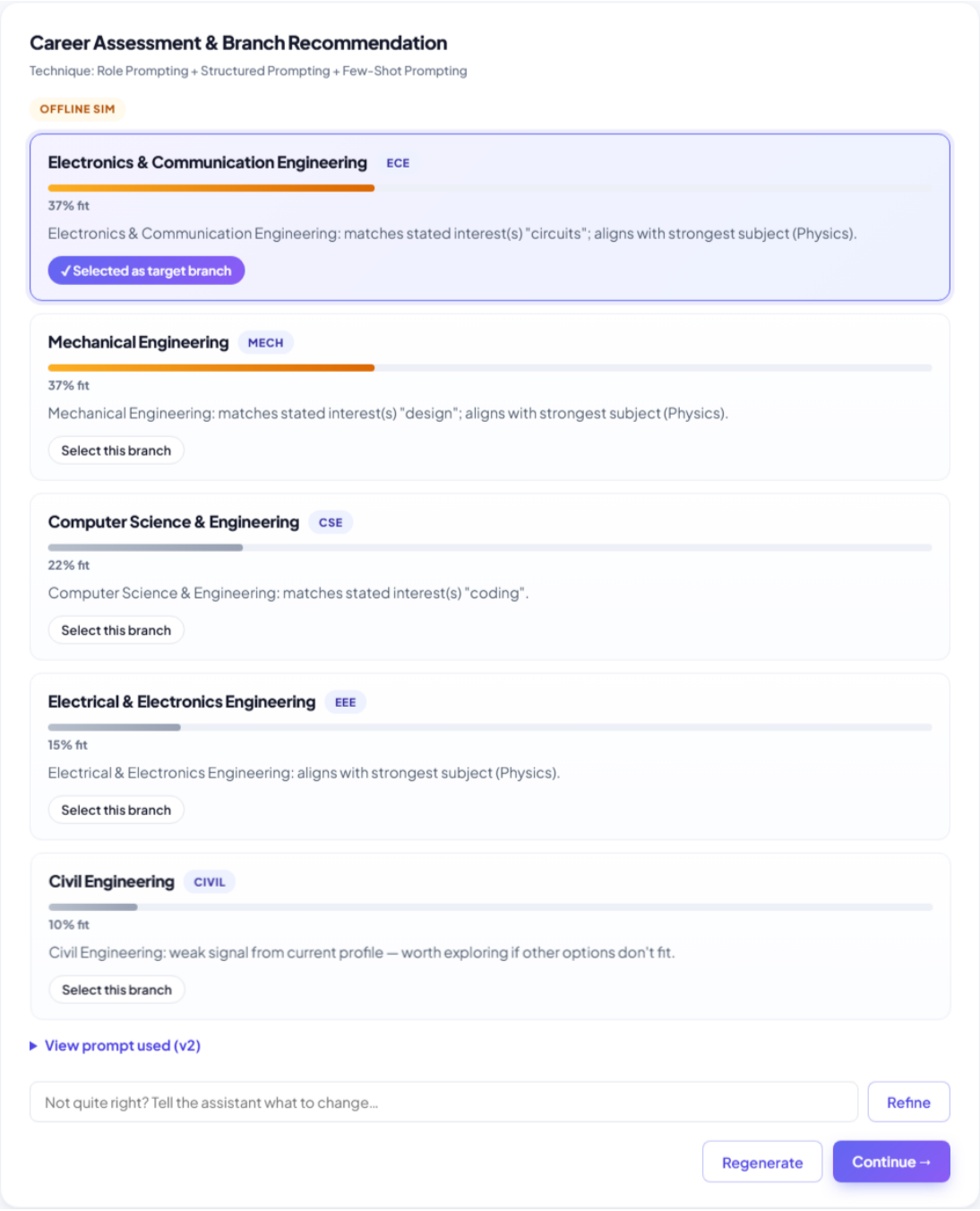

Figure 3: Assistant tab – Career Assessment stage output with reasoning, branch picker, and prompt-view toggle.

College Comparison

Technique: Structured Prompting (fixed multi-parameter rubric) + Template Prompting

OFFLINE SIM

#1 IIT Delhi — 94
IIT · Delhi · NBA + NAAC A++
- ✓ High placement rate (95%)
- ✓ Strong average package (~₹22 LPA)
- ✓ Very High research activity
- ✓ Strong student review score (4.6/5)

☆ Save to shortlist

#2 IIT Bombay — 93
IIT · Maharashtra · NBA + NAAC A++
- ✓ High placement rate (95%)
- ✓ Strong average package (~₹21 LPA)
- ✓ Strong student review score (4.6/5)

☆ Save to shortlist

#3 NIT Tiruchirappalli — 82
NIT · Tamil Nadu · NBA + NAAC A++
- ✓ High placement rate (90%)
- ✓ High research activity
- ✓ Strong student review score (4.4/5)

☆ Save to shortlist

#4 NIT Warangal — 81
NIT · Telangana · NBA + NAAC A++
- ✓ High research activity
- ✓ Strong student review score (4.4/5)

☆ Save to shortlist

#5 COEP Technological University, Pune — 76
State Govt. · Maharashtra · NBA + NAAC A
- ⚠ Lower placement rate (82%) than top options

☆ Save to shortlist

#6 VIT Vellore — 75
Private (Deemed) · Tamil Nadu · NBA + NAAC A++

☆ Save to shortlist

#7 BITS Pilani — 54
Private (Deemed) · Rajasthan · NAAC A

OVER STATED BUDGET
- ✓ High placement rate (92%)
- ✓ Strong average package (~₹18 LPA)
- ✓ High research activity
- ✓ Strong student review score (4.5/5)
- ⚠ Fees (₹520000/yr) exceed stated budget (₹250000/yr)

☆ Save to shortlist

#8 Manipal Institute of Technology — 46
Private (Deemed) · Karnataka · NBA + NAAC A++

OVER STATED BUDGET
- ⚠ Fees (₹480000/yr) exceed stated budget (₹250000/yr)
- ⚠ Lower placement rate (83%) than top options

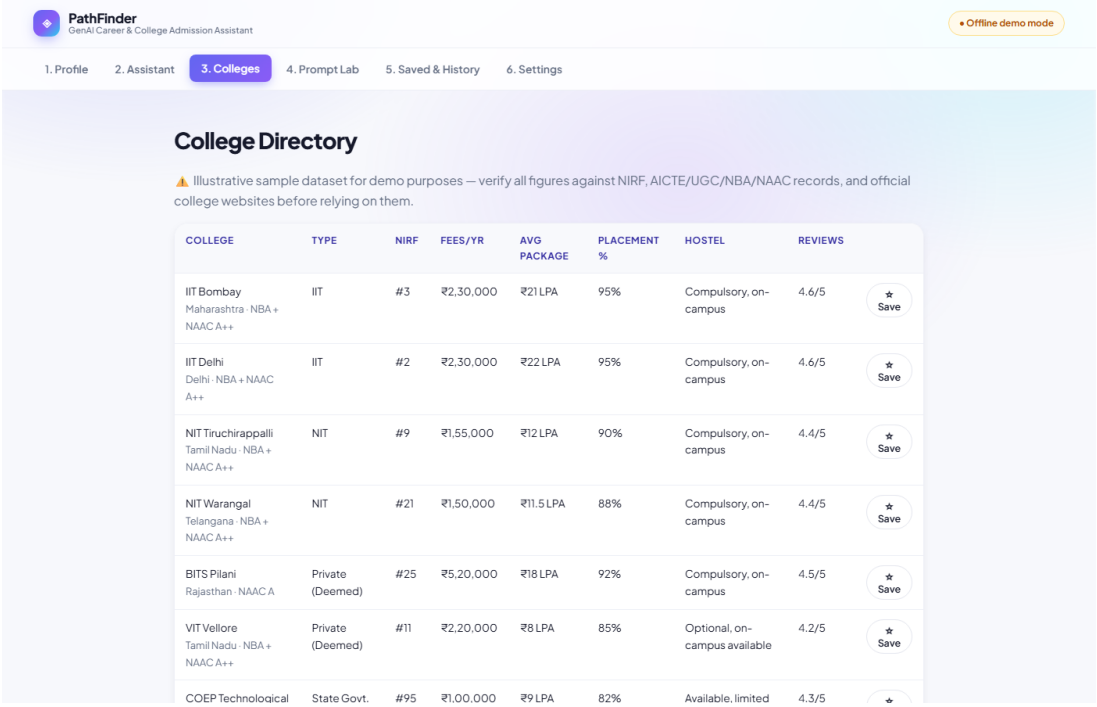

Figure 5: College Directory tab with the illustrative-data disclaimer and save-to-shortlist controls.

7 Live-Mode Failure Modes Found in Testing

The offline simulator behaved correctly from the start, but switching the same prompts to Live mode against the real Claude API surfaced four distinct integration failures. They are documented here rather than quietly patched, because each one is a general lesson about deploying LLM output into a structured UI – and because the last one is a failure of *my own testing method*, not of the model.

7.1 Symptom-by-symptom

Failure	Cause	Fix
Card renders blank	Model wrapped valid JSON in a `"‘json` markdown fence despite a JSON-only instruction; `JSON.parse` threw and the failure was swallowed	`parseModelJson()` strips fences, then falls back to extracting the first `{...}` block; parse failure now shows the raw text in a banner instead of nothing
JSON cut off mid-string	College Comparison output for 8 colleges exceeded `max_tokens` (1200), truncating into invalid JSON	Raised the cap, *and* capped output length at the prompt level (max 3 strengths, 2 tradeoffs, one $\leq$25-word reasoning per college) so length cannot run away with candidate count
Truncation misreported	A cut-off response was reported with the generic “couldn’t parse” message	Threaded the API’s `stop_reason` through so genuine truncation says so explicitly

Failure	Cause	Fix
Whole stages empty	*See §7.2 – the instructive one*	`stageCtx()` corrected + a guard that refuses to send the prompt

7.2 The reference-data bug – and why my testing missed it

Two stages build their prompts as `buildV2(profile, referenceData)`, but `stageCtx()` returned only `{ profile }` for them. `JSON.stringify(undefined)` yields `undefined`, which interpolated into the template as literal text:

```
## ENTRANCE_EXAMS reference (do not invent exams outside this list)
undefined
```

The model was instructed to choose only from that list, the list was the word "undefined", so it correctly returned `{"recommended_exams": []}` – an empty card. **The model behaved perfectly; the prompt was broken.** The same defect left Career Prospects with no branch data (rendering `NaN` for salary) and degraded the chained Admission Roadmap, which consumes the exam stage's output.

The reason this survived earlier testing is the more important finding: *every test had been run in offline mode, and the offline simulators read `ENTRANCE_EXAMS/BRANCHES` directly and never touch the prompt string at all.* The test suite was structurally incapable of exercising the broken path. A prompt-driven system needs tests that assert on *the generated prompt text*, not only on rendered output.

7.3 Resulting safeguards

- **Fail loudly, not silently.** `runStage()` now throws if a builder declaring a reference-data parameter is invoked without one, rather than shipping a prompt containing "undefined" to the API.
- **Prompt-text auditing.** A harness builds the V2 prompt for all 8 stages exactly as the app does and asserts none contains a stray `undefined`, `[object Object]`, or an empty reference block. All 8 pass.
- **Tolerate field-name drift.** Live models occasionally rename schema keys (`exam` for `code`, `reason` for `why_relevant`). Renderers resolve through a list of accepted aliases and omit absent fields rather than printing "undefined"; a renderer suite covers 14 cases including deliberately renamed keys.
- **Never render nothing.** If a stage renderer produces visually empty markup for any reason, the UI falls back to a labelled raw-data view – a silent blank card in front of a student (or an examiner) is the worst possible failure.

This is also the honest answer to the brief's "reliability" criterion: structured-output prompting is not a guarantee. A production system needs schema validation on every response, and the graceful-degradation path matters as much as the happy path.

8 Evaluation of AI-Generated Recommendations

Figure 6: Admission Roadmap – the chained stage (§3) rendered as a gradient timeline, with document checklist and contingency notes as side panels.

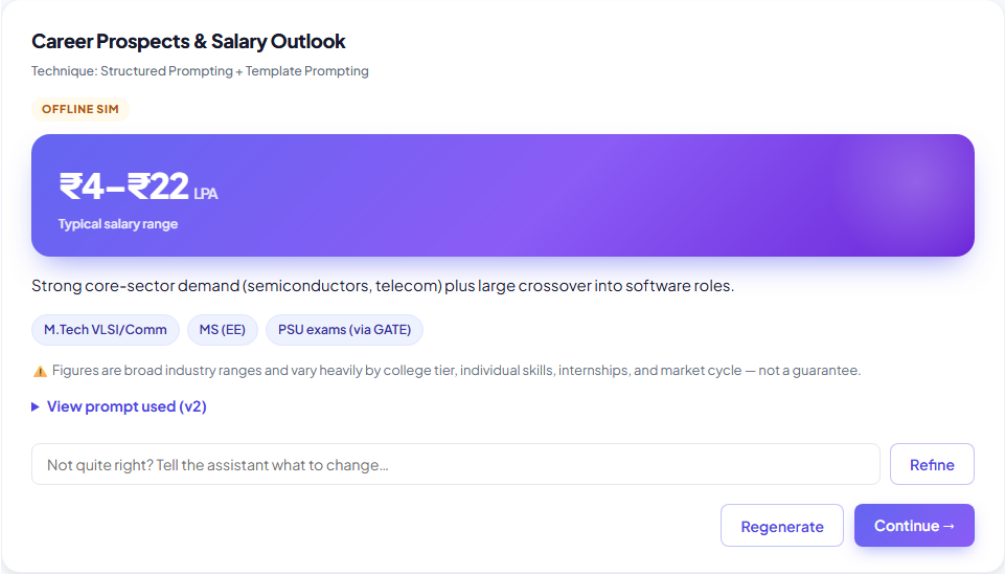

Figure 7: Career Prospects – salary shown as a caveated *range* in a stat callout, never a single guaranteed figure (§9).

Criterion	Design response & honest assessment
Accuracy	Structured prompts constrain colleges/exams/scholarships to a closed reference list (§9) so the model cannot invent entities. Numeric *values* (fees, packages) are only as accurate as the underlying illustrative dataset – not independently verified against live sources in this prototype, which is exactly why §11 exists as a required, not optional, step.
Transparency	Every single AI response in the app ships with a visible "View prompt used" panel and a Live/Offline mode badge – nothing is hidden.
Explainability	Every schema mandates a `reasoning` (or `why_*`) field grounded in the student's own inputs (demonstrated in §4); V1's absence of this requirement is used deliberately to show what explainability looks like when it's *missing*.
Personalization	All eight prompts are template-filled from the specific student profile (interests, budget, state, category); College Comparison explicitly flags budget mismatches per student rather than giving one generic ranking.
Completeness	Profile Intake & Clarification stage runs first and refuses to proceed silently – it asks up to 3 targeted questions when required fields are missing (demonstrated by `out_intake_incomplete.json` in `report/samples/`).

Criterion	Design response & honest assessment
Consistency	Fixed JSON schemas per stage keep output shape stable across runs; the offline simulator is fully deterministic. Live-mode consistency proved weaker than the schema alone implies – the model varied its wrapping (code fences), its verbosity (enough to overrun the token budget), and occasionally its key names between runs (§7), which is precisely why the parsing and rendering layers are now written defensively. Low-temperature settings for factual stages remain recommended but unimplemented.
Fairness & bias	Gender/category/income fields are collected only for the scholarship-matching stage, are optional, and never influence branch or college recommendations (verified by code inspection of `stageCtx()` – these fields are absent from every context object except `scholarship_finder`). Residual bias risk: the branch-keyword mapping in `data.js` reflects the author's own judgement of which interests map to which branch and was not validated against a counsellor or larger dataset.
Reliability	Offline mode guarantees the app functions with zero external dependency. Live mode initially failed in four distinct ways (§7) – fenced JSON, token truncation, a broken reference-data hand-off, and field-name drift – all now fixed and guarded, with a never-render-blank fallback. The honest conclusion is that structured-output prompting is not self-guaranteeing: a production system needs schema validation on every response.
Ethical AI usage	System prompts explicitly instruct the assistant never to fabricate specific college/exam/scholarship facts and to say so when uncertain; salary/outlook figures are always presented as ranges with caveats, never guarantees; sensitive personal fields are clearly optional and scoped.

9 Minimizing Hallucinations

Strategies actually implemented in the running prompts (not just discussed in the abstract):

- **Closed reference lists.** College Comparison, Exam Recommendation, and Scholarship Matching prompts each embed the full candidate dataset in the prompt and explicitly instruct: *"do not add colleges/exams/schemes outside this list."*
- **Forced structured output + refusal path.** Every schema includes `needs_clarification` and `missing_or_ambiguous` fields so the model has an explicit, in-schema way to say "I don't have enough information" instead of confabulating.
- **Verification hand-off fields.** College Comparison output includes `verify_before_deciding` (e.g., "current NIRF ranking", "latest official placement report"); Counselling Guidance output includes `official_sources`; the system prompt for Counselling Guidance explicitly forbids stating specific current-year cutoff numbers.
- **Range, not point, claims.** Career Prospects always returns a `salary_range_lpa` plus a `caveats` array, never a single figure presented as certain.

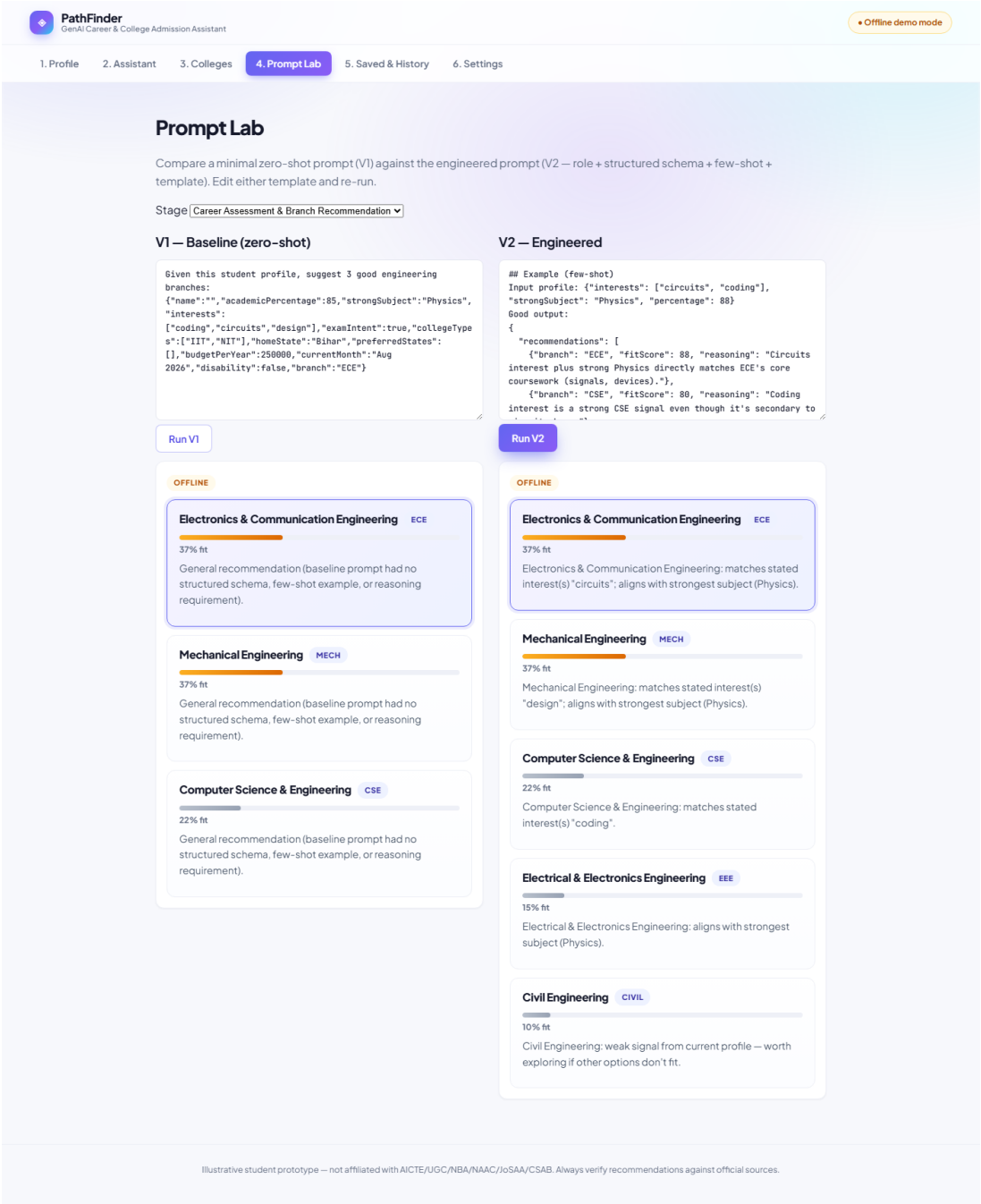

Figure 8: Prompt Lab – V1 (zero-shot) and V2 (engineered) templates side by side. V1's output carries no grounded reasoning; V2's cites the specific interest and subject behind each recommendation.

- **Not yet implemented (future work):** temperature=0 for factual stages in Live mode, and a retrieval step against a real, regularly-refreshed NIRF/AICTE dataset instead of the static illustrative one.

10 Protecting Student Data Privacy

- All profile data, shortlist, and history persist only in the browser's `localStorage` – there is no backend and no server-side database in this prototype, so there is nothing to breach server-side.
- The Anthropic API key (Live mode) is likewise stored only in `localStorage` and is sent only to `api.anthropic.com` at the moment a stage is run – never to any third party, never logged.
- Sensitive optional fields (gender, category, income band, disability) are visually separated in the Profile form under a collapsed "optional, for scholarship matching only" section, and are structurally excluded from every other stage's context (§8, Fairness row).
- A "Clear all data" control in Settings wipes everything from the browser in one action.
- **Production gap, stated honestly:** a real deployment should not ask each student to supply their own API key. It needs a server-side proxy holding the key, per-user authentication, encryption at rest for any persisted profile data, a documented data-retention/deletion policy, and compliance review (India's DPDP Act, 2023, for a system handling minors' data) – none of which a client-only prototype can or should attempt to fake.

11 Validation Against Official Sources

Because this environment has no access to live AICTE / UGC / NBA / NAAC / NIRF / JoSAA / CSAB feeds, the dataset in `data.js` is explicitly labelled illustrative rather than presented as verified. The validation workflow a real deployment must run before trusting any figure shown to a student:

1. **Institutional accreditation & approval** – cross-check every listed college against the current AICTE-approved institutions list and UGC/NBA/NAAC accreditation records before display.
2. **Rankings** – refresh NIRF Engineering rankings from the official NIRF publication each cycle; timestamp every ranking shown with a "last verified on" date.
3. **Counselling data** – seat matrices, cutoffs, and round dates must be pulled from JoSAA/CSAB or the relevant state admission committee at the start of each admission cycle; the app's Counselling Guidance stage is deliberately designed to explain *process*, never state specific cutoff numbers, for exactly this reason.
4. **Fees, placements, and reviews** – sourced from official college websites and their published, audited placement reports, not third-party aggregators, with a visible source citation per figure.
5. **Recurring re-validation** – given yearly churn in cutoffs, fees, and placement stats, any production version needs an annual data-refresh pipeline, not a one-time import.

12 Limitations & Future Work

- The offline simulator demonstrates the prompt design's intended behaviour deterministically but is not itself a language model; genuine reasoning quality, robustness to unusual phrasing, and multi-

turn conversational nuance can only be assessed in Live mode with a real API key. As §7 shows, the simulator can also *mask* real integration defects, so it should be treated as a demo affordance rather than as evidence the system works.

- There is no schema validation on live responses – parsing is defensive and renderers tolerate aliases, but nothing formally validates a response against the declared schema and re-prompts on mismatch. That is the single most valuable next engineering step.
- The prompt-text audit and renderer suite are standalone scripts run manually, not a wired-up CI test suite.
- The reference dataset (8 colleges, 7 exams, 6 scholarship schemes) is a small illustrative sample, not the full national landscape, and is not currently re-validated on any schedule.
- No authentication, multi-device sync, or server persistence exists; the app is single-browser, single-device by design for this prototype.
- Only one model provider (Anthropic Claude) is integrated; a production system may want provider redundancy.
- Free-text refine/feedback input has no moderation layer – acceptable for a course prototype, not for a public deployment.
- Accessibility (screen-reader labelling, keyboard navigation audit) and multi-language support (most target students are more comfortable in a regional language for this decision) were out of scope here but matter for real-world reach and fairness. The hero scene does honour `prefers-reduced-motion`, but that is one narrow slice of accessibility, not a substitute for an audit.

13 Conclusion

PathFinder demonstrates the full guidance journey the assignment specifies – profile intake with ambiguity handling, chained multi-stage recommendations, multi-parameter college comparison, exam and scholarship matching, roadmap generation, and counselling explanation – built on a documented library of role, structured, template, few-shot, chaining, and iterative-refinement prompts, with every recommendation traceable to the exact prompt that produced it.

Its most important design choice is refusing to pretend. Illustrative data is labelled as such; salary and placement claims are ranges with caveats; the evaluation and validation sections report real gaps rather than assumed compliance. §7 extends that to the engineering itself: four live-integration failures are documented with their causes – including one where a malformed prompt of my own making sent the word “undefined” to the model, and the model’s empty answer was the correct response to it. The most transferable lesson from building this is that in a prompt-driven system the prompt is executable code and deserves to be tested as such: the reason that defect survived was a test suite that only ever exercised the offline path and never asserted on the generated prompt text.